Precision@k Is a Great Target and a Dangerous Judge: Model Selection for Fixed-Budget Top-k Problems

Alejandro Gómez Ruiz[1]

[1]Independent Researcher , alejandrogomez1997@gmail.com

July 15, 2026

Abstract

In many operational problems — energy-fraud inspection, tax audits, retention campaigns, medical call-backs — a model ranks a population and a team acts only on the top k, where k is a fixed external budget. The natural objective is *precision@k*. It is well known that precision@k cannot serve as a training loss for gradient-based learners, because it is piecewise-constant and non-differentiable (and, for tree ensembles, because it does not decompose into per-node gains); we make the mechanism concrete with a worked gradient-boosting step. We then ask a less-examined question: should precision@k be the metric used for *model selection*? Across 93 imbalanced datasets (90 real, 3 synthetic) with the budget K swept over two orders of magnitude, and across three model families (gradient boosting, random forests, logistic regression), we find that selecting a model by validation *average precision* yields *significantly better* held-out precision@k than selecting by precision@k itself (paired Wilcoxon, $p < 1.2 \times 10^{-3}$ in every family): optimizing selection for the exact target metric is a losing move, a manifestation of selection-induced optimism (the winner's curse). Log-loss is an equally good selector for gradient boosting but its edge is model-dependent, whereas average precision is robust; ROC-AUC is never better than precision@k. We further show that selection difficulty is governed *not* by any scale-free ratio such as n_{pos}/K (Spearman $|\rho| \approx 0.19$), but by the absolute *count* of true positives that land in the budget ($|\rho| \approx 0.43$); its model-free proxy $\min(K, n_{\text{pos}})$, knowable before training, tracks that count almost perfectly ($\rho = 0.94$). We give practical recommendations and release all code.

1 Introduction

Consider a utility's fraud unit whose crew can inspect a fixed number of installations per month. A meter ranked below the cutoff is never inspected, so calibration or ranking quality in the bulk of the distribution is operationally irrelevant; only the composition of the top k matters. We call this a *fixed-budget top-k problem*: score a population, act on the top k, where k is imposed from outside the model (crew size, budget, legal quota). The natural success metric is

$$\text{precision@}k = \frac{\#\{\text{true positives among the top-}k\text{ scored items}\}}{k}. \tag{1}$$

Such problems are ubiquitous (audits, marketing, triage, moderation, lead scoring).

As *background*, we first recall the well-known reason precision@k cannot be a *training* loss for gradient-based learners — it is piecewise-constant with a zero gradient — and make it concrete with a worked gradient-boosting step (Section 2). This is not a contribution; it is the setup. The

interesting question is *model selection*, where precision@k *is* admissible (we only need to compute it, not differentiate it). Here the natural instinct is compelling: since we want the model with the highest *test* precision@k, we should keep the model with the highest *validation* precision@k. Our contributions are about whether that instinct holds.

(1) Empirically, across 93 imbalanced datasets and three model families (gradient boosting, random forests, logistic regression), it does not: selecting by *average precision* yields significantly better held-out precision@k than selecting by precision@k itself, in every family (Section 5). **(2)** We explain why. Precision@k is estimated from only the k scored items in the budget, so it is a high-variance selection criterion; taking the maximum over many configurations then overfits that noise — the model-selection overfitting of Cawley and Talbot [3] and Varma and Simon [12], here specialised to a top-k metric. Average precision and log-loss instead aggregate over the *entire* validation set ($n_{\text{val}} \gg k$ points), making them far more stable estimators; that stability outweighs their looser alignment with the top-k target. **(3)** We characterise *what* governs the trade-off: not a scale-free ratio such as n_{pos}/K, but the absolute number of true positives inside the budget, whose a-priori, model-free proxy $\min(K, n_{\text{pos}})$ is knowable before training.

2 Background: why precision@k cannot be a training loss

Precision@k depends on the scores only through the *set* of top-k items. An infinitesimal change to any single score almost never changes that set, so the metric is locally constant and its gradient is zero almost everywhere; it changes only at measure-zero boundaries where two items swap across the cutoff, in jumps of $1/k$ (Fig. 1). Gradient boosting fits each new tree to the negative gradient of the loss (the pseudo-residuals); with precision@k every pseudo-residual is zero, so the ensemble never moves.

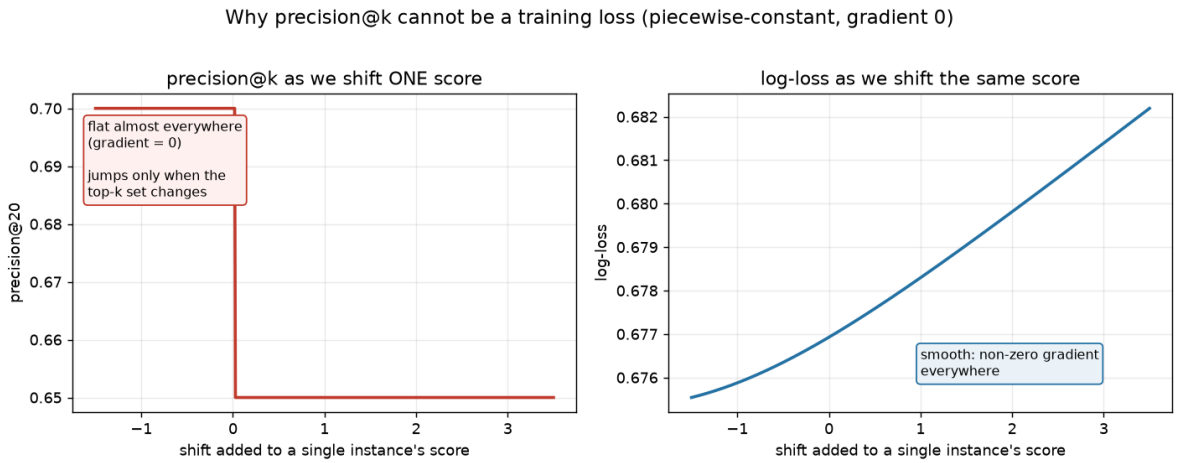

Figure 1: Sliding one item's score: precision@k is piecewise-constant with a single cliff (gradient 0 almost everywhere), whereas log-loss is smooth with an informative gradient.

Table 1 makes it concrete for $k = 3$. For log-loss with a sigmoid, the gradient w.r.t. each logit is $p - y$: false positives ranked high get large positive gradients ("push down"), missed positives get negative gradients ("push up"). Every row receives a signed instruction. Precision@3, by contrast, gives zero for every row: the only helpful move (lifting D above C, changing the top-3 to $\{A, B, D\}$ and precision@3 from 1/3 to 2/3) is a *finite* jump the gradient cannot see. Differentiable surrogates

for top-k losses exist [1, 7], as does the LambdaMART device of *defining* metric-aware gradients [2]; for most systems the pragmatic recipe remains: train on log-loss, evaluate on precision@k.

The zero-gradient argument is specific to *gradient-based* learners (logistic and linear regression, neural networks, gradient boosting). Non-gradient learners such as decision trees and random forests fail to admit precision@k as a training objective for a *different* reason: they are grown by greedily optimising a *local*, per-node impurity criterion (e.g. Gini), whereas precision@k is a *global*, ranking-based, k-dependent quantity that does not decompose into per-node gains. Either way, precision@k is used only for evaluation and selection; accordingly, in our experiments no model is trained on it (gradient boosting and logistic regression minimise log-loss, random forests minimise Gini impurity).

Table 1: One boosting step, budget $k = 3$. Top-3 by score are A, B, C, so precision@3 $= 1/3$. Log-loss gradient $= p - y$; precision@3 gradient $= 0$ everywhere.

item	score p	label y	log-loss grad. $(p - y)$	precision@3 grad.
A	0.92	fraud (1)	-0.08	0
B	0.85	legit (0)	$+0.85$	0
C	0.80	legit (0)	$+0.80$	0
D	0.78	fraud (1)	-0.22	0
E	0.60	fraud (1)	-0.40	0
F	0.55	legit (0)	$+0.55$	0
G	0.30	fraud (1)	-0.70	0
H	0.10	legit (0)	$+0.10$	0

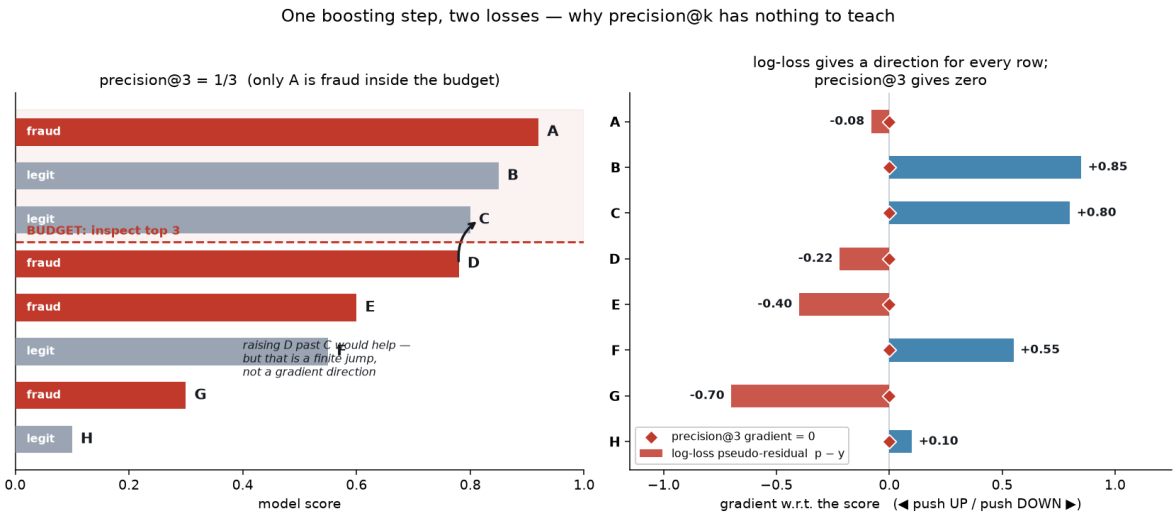

Figure 2: The same step visualised. Right: log-loss assigns a direction to every row; precision@3 assigns zero. Note log-loss already pushes C down and D up — the swap that would raise precision@3.

3 Precision@k as a selection metric: two opposing forces

Evaluation — early stopping and choosing among trained configurations — only requires *computing* the metric, so precision@k is admissible there. Two forces pull in opposite directions.

Alignment (favouring precision@k). Log-loss is a global objective: it rewards calibration across the whole distribution, not the top-k composition. A model can lower average log-loss by improving many mid-ranked cases while letting a few clear positives slip out of the budget — better by log-loss, worse at the operational objective. Only precision@k detects this.

Stability (against precision@k). Precision@k is a proportion estimated from only the k items in the budget, so its standard error scales like $\sqrt{p(1-p)/k}$ — it is a high-variance criterion. Selecting the maximum over many configurations on one validation set then partly selects noise — the winner's curse [3] — so the chosen model generalises worse and the reported number is optimistically biased. Average precision and log-loss are, by contrast, aggregates over the *whole* validation set: average precision integrates precision along the entire precision–recall curve [4], and log-loss averages a per-example term over all $n_{\text{val}} \gg k$ points. Their estimation error therefore shrinks with n_{val}, not with the tiny k, so they are far more stable selection criteria — less aligned with the exact top-k target, but much less prone to the winner's curse. Which force dominates is an empirical question.

4 Experimental protocol

Datasets. 90 real binary imbalanced datasets — the full `imbalanced-learn` benchmark [9] plus a de-duplicated pull from OpenML [11] (fraud, credit default, software-defect, churn, bankruptcy, medical screening, satellite/pulsar anomalies), positive rates from 35% to 0.7% — plus 3 synthetic datasets. Large sets are subsampled to 30k rows.

Model bank. Per dataset we split 50/25/25 (train/validation/test, stratified) and train a bank of C configurations with random hyperparameters. The main study uses $C = 40$ LightGBM [8] configurations (trained on log-loss); to show the conclusion is not learner-specific, Section 5 repeats the *entire* protocol with two further families — random forests and L_2-regularised logistic regression, $C = 25$ each (scikit-learn [10]). Each learner is trained with its own standard objective (log-loss for boosting and logistic regression, Gini impurity for random forests); precision@k is never a training target, only an evaluation/selection metric.

Budget sweep. Throughout, n_{pos} denotes the number of positives in the *evaluation cohort* being ranked (the test split), *not* the whole dataset — in the fraud analogy, the test split is the monthly batch of installations, n_{pos} is the frauds present in that batch, and K is how many of them you can inspect. We set K to hit target ratios $n_{\text{pos}}/K \in \{0.25, 0.5, 1, 2, 4, 8, 16\}$: a value of 16 means the cohort contains 16× more positives than the budget (a tiny budget — the typical fraud regime), whereas 0.25 means the budget is four times larger than *all* the positives. This spans budgets far larger and far smaller than the positive pool. Note this ratio is distinct from the *positives that land inside* the budget (which is at most K and drives the results below); n_{pos}/K compares the budget to the whole cohort's positive pool.

Selection under noise. We bootstrap [6] the validation set 120 times; for each metric $m \in$ {precision@k, average precision (AP) [4], ROC-AUC, log-loss} we select $\arg\max_c m$ and record that configuration's *true* quality, precision@k on the held-out test pool.

Normalized regret. To pool heterogeneous datasets we report, per (dataset, K),

$$\text{regret}(m) = \frac{q^\star - \bar{q}_m}{q^\star - \bar{q}_{\text{avg}}}, \tag{2}$$

where $q^\star$ is the best test precision@k in the bank (oracle), $\bar{q}_m$ the mean test quality selected by m, and $\bar{q}_{\text{avg}}$ the average model's quality. 0 means oracle selection, 1 means no better than picking at random. We keep the 322 (of 625) cases with a non-trivial gap ($q^\star - \bar{q}_{\text{avg}} \geq 0.02$). Following Demšar [5] we compare metrics with the paired Wilcoxon signed-rank test.

5 Results

Log-loss and AP beat precision@k, significantly. Table 2 and Fig. 3 summarise the pooled comparison. Log-loss and AP tie for the lowest mean regret and are statistically indistinguishable from each other ($p = 0.53$); each beats precision@k with $p < 2 \times 10^{-3}$. AUC is no better than precision@k ($p = 0.85$). In words: *selecting on the metric you ultimately care about gives you a worse model, on that same metric, than selecting on a more stable proxy.*

Table 2: Model-selection metrics pooled over 322 (dataset, budget) cases. Mean normalized regret (lower is better), win-rate (fraction of cases where the metric picked the single best model), and paired Wilcoxon p-value versus precision@k.

selection metric	mean regret ↓	win-rate	Wilcoxon p vs. P@k
log-loss	**0.714**	**34%**	6.1×10^{-4}
average precision	0.719	21%	1.2×10^{-3}
ROC-AUC	0.770	20%	0.85 (n.s.)
precision@k	0.790	25%	—

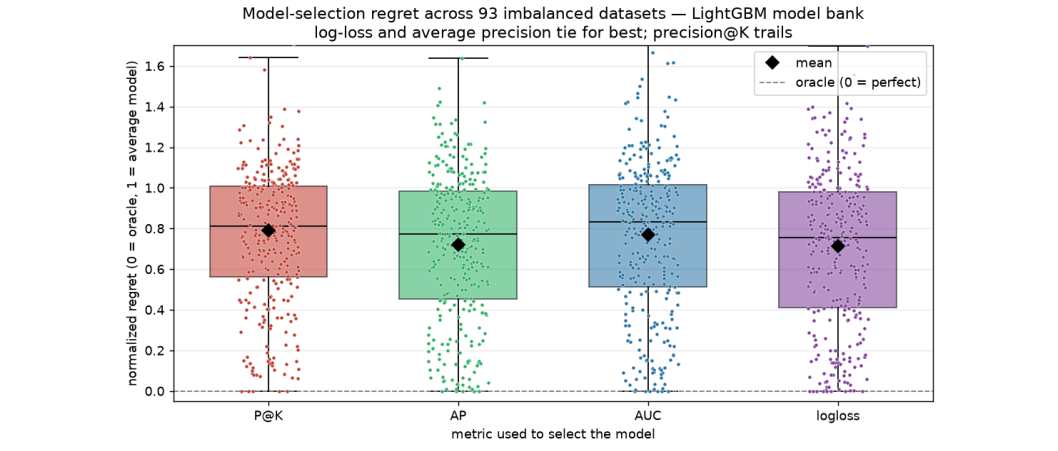

Figure 3: Distribution of normalized regret per selection metric across the 322 cases; diamonds mark means. Log-loss and average precision lead; precision@k trails.

It is a count, not a ratio. One might expect precision@k to be safe whenever the budget is large relative to the positives (large n_{pos}/K). It is not: that ratio barely orders regret (Spearman $|\rho| = 0.19$). What orders it is the *absolute number of true positives inside the budget* ($\approx$ precision $\times K$; $|\rho| = 0.43$), Fig. 4. A proportion's reliability is set by the number of successes behind it — a count, not a fraction — so two problems with the same ratio behave differently if one is ten times larger.

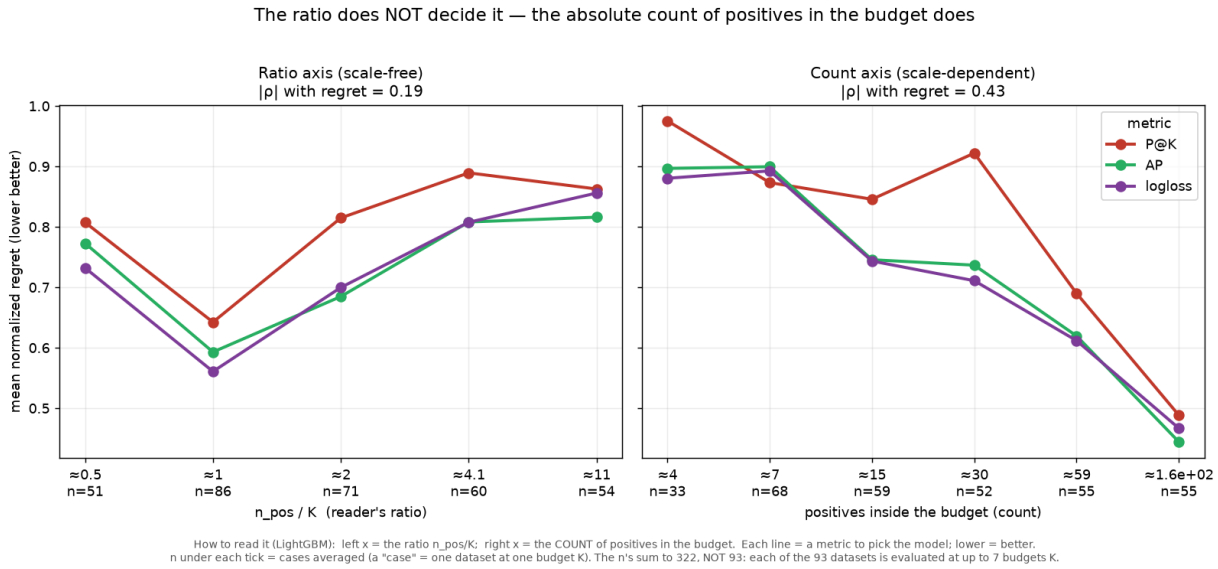

Figure 4: Left: the scale-free ratio n_{pos}/K barely orders regret. Right: the absolute count of positives inside the budget does. n = number of (dataset, budget) cases per bin.

The count is knowable a priori. The achieved count (precision $\times K$) depends on the model, but its model-free upper bound $\min(K, n_{\text{pos}})$ — the most positives that could possibly land in the budget — predicts almost as well ($|\rho| = 0.38$) and tracks the achieved count almost perfectly ($\rho = 0.94$), Fig. 5. So the deciding quantity is available before training; for the exact value, count the true positives in a single cheap baseline's top-k.

The reported number is inflated too. Consistent with the winner's curse, validation precision@k overstates true precision in proportion to how few positives sit in the budget — by tens of percent, occasionally more than 2× — so it is also an optimistic estimate of production performance (Fig. 6).

The result holds across model families. The experiments above use LightGBM. To check that the conclusion is not an artefact of one learner, we repeat the entire selection study with two very different families: random forests and L_2-regularised logistic regression (25 configurations each; datasets subsampled to 15k rows). Table 3 and Fig. 7 show the same qualitative picture in all three families: **average precision selects significantly better models than precision@k everywhere** ($p \leq 1.2 \times 10^{-3}$), while ROC-AUC is never better than precision@k. Log-loss is tied-best for gradient boosting but does *not* beat precision@k for the other two families — its advantage is model-dependent, whereas average precision's is robust.

The count-driver result also transfers: Fig. 8 reproduces, for each family, the fall of regret with the number of positives inside the budget, with average precision at or below precision@k

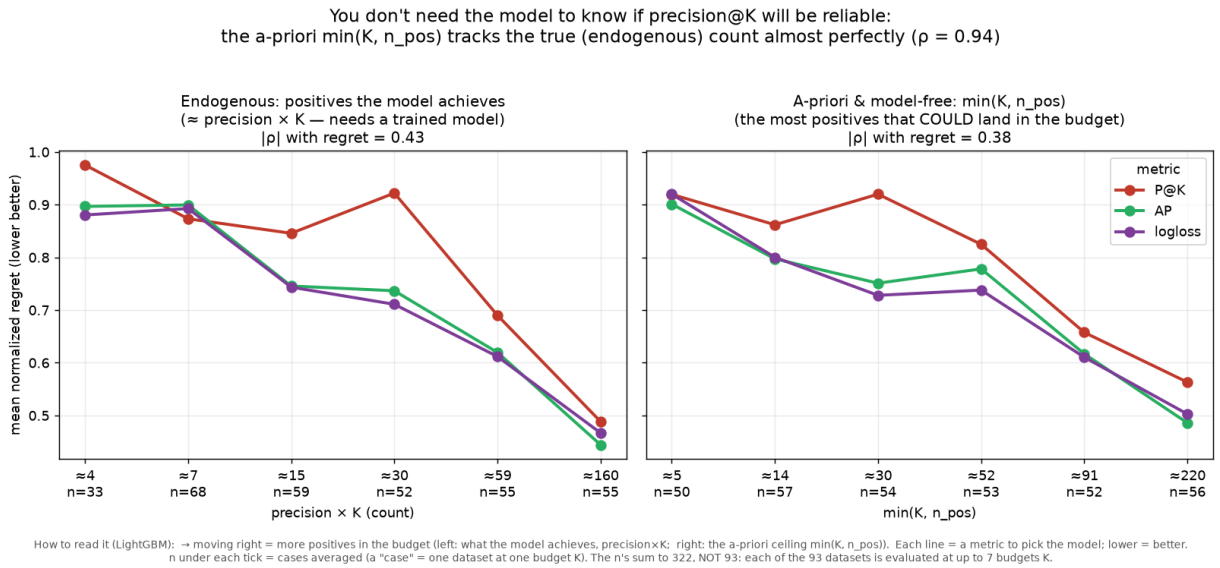

Figure 5: The endogenous count precision $\times K$ (left) and its a-priori, model-free proxy $\min(K, n_{\text{pos}})$ (right) order regret almost identically ($\rho = 0.94$ between them).

Table 3: Robustness across model families. Mean normalized regret (lower is better) and paired Wilcoxon p-value versus precision@k. Average precision is significantly better than precision@k in every family; log-loss only for LightGBM; ROC-AUC never.

	precision@k	average precision	log-loss	ROC-AUC
LightGBM (n=322)	0.790	0.719 (p=1.2×10^{-3})	**0.714** (6.1×10^{-4})	0.770 (n.s.)
Random forest (n=329)	0.710	**0.663** (5.7×10^{-4})	0.746 (n.s.)	0.717 (n.s.)
Logistic reg. (n=288)	0.750	**0.631** (2.9×10^{-4})	0.697 (n.s.)	0.839 (worse)

throughout.

The two other pieces of the fine analysis also replicate across families. The scale-free ratio n_{pos}/K fails to order regret in *any* family (Fig. 9: the curves are non-monotone and tangled), whereas the a-priori, model-free proxy $\min(K, n_{\text{pos}})$ orders it cleanly and monotonically in all three (Fig. 10). The count of positives in the budget — not a ratio — is what governs selection difficulty, regardless of the learner.

6 Discussion and recommendations

Keep precision@k as the target and the headline evaluation metric, but not as the training loss and not, by default, as the selection metric. For selection, **default to average precision** — it was the only metric to beat precision@k significantly in every model family; log-loss is an equally good choice for gradient boosting but not in general. Use precision@k only as a tie-breaker. The single lever that matters is knowable in advance: $\min(K, n_{\text{pos}})$, the number of positives that can land in the budget. When it is small, no selection metric is reliable — invest in more labelled positives or a larger budget, not a cleverer metric. Finally, never quote raw validation precision@k as expected production performance when the budget is positive-poor; report a nested/held-out estimate.

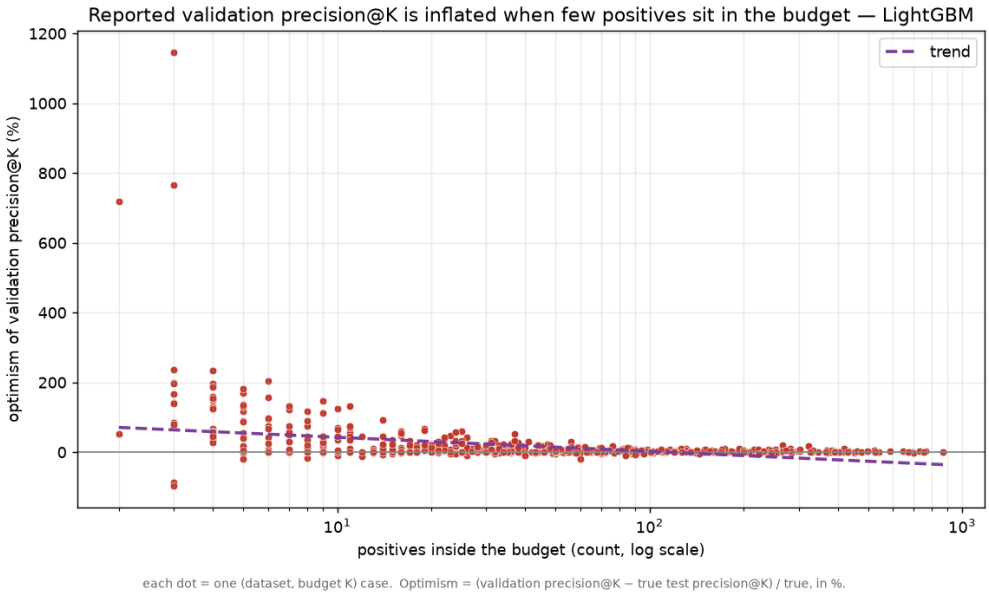

Figure 6: Optimism of validation precision@k (over-statement of the selected model's true precision) versus the number of positives in the budget.

7 Limitations

We study three model families (gradient boosting, random forests, logistic regression) but a fixed model bank, isolating selection noise but not early stopping (where the same variance argument applies). Ground-truth precision@k on small datasets is itself estimated with noise, mitigated by pooling and by robust statistics (rank, win-rate, signed-rank tests). Results are statements about *expected* selected quality. Extending to deep models, and to explicit top-k surrogate training, is future work.

8 Conclusion

The metric you optimize and the metric you select with need not be the same. For fixed-budget top-k problems *on imbalanced data* — the regime studied here, and the one in which these problems arise — selecting on *average precision* robustly beats selecting on the noisy target metric itself, across 93 imbalanced datasets and three model families, and the deciding factor for how much any of this matters is a plain count, how many positives your budget will contain, not a ratio of rates. Whether the effect persists on near-balanced data, where the top-k slice is less positive-starved, is left to future work.

Reproducibility. All code, the dataset list, and figures are released at `https://github.com/alejandrogomez97/precision-at-k-model-selection` and run from a fixed seed.

References

[1] Stephen Boyd, Corinna Cortes, Mehryar Mohri, and Ana Radovanovic. Accuracy at the top. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, 2012.

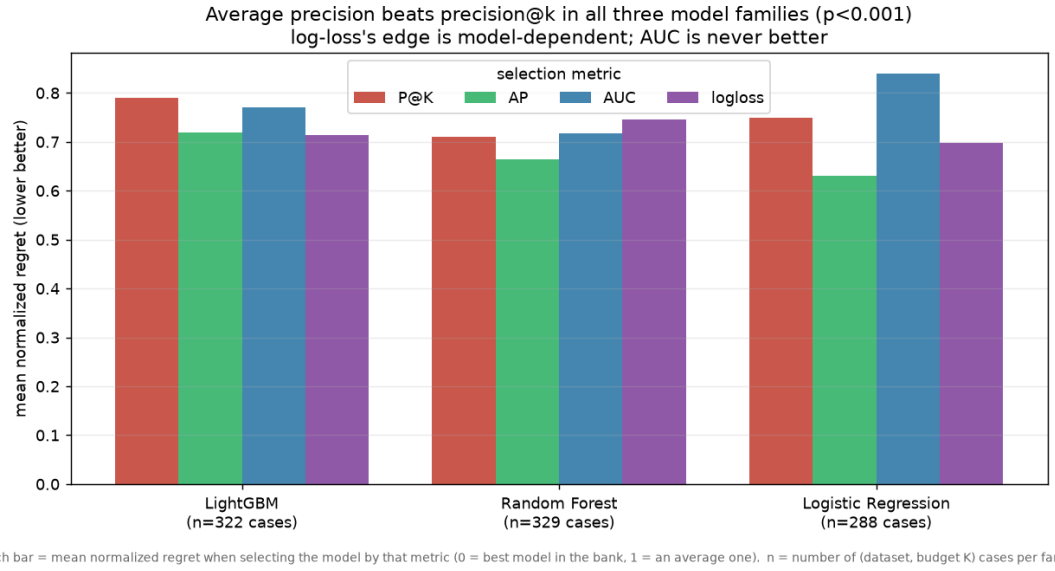

Figure 7: Mean normalized regret per selection metric, grouped by model family. Average precision (green) is lowest in all three; log-loss (purple) leads only for gradient boosting.

[2] Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.

[3] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.

[4] Jesse Davis and Mark Goadrich. The relationship between precision-recall and ROC curves. In *International Conference on Machine Learning (ICML)*, 2006.

[5] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

[6] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1994.

[7] Purushottam Kar, Harikrishna Narasimhan, and Prateek Jain. Surrogate functions for maximizing precision at the top. In *International Conference on Machine Learning (ICML)*, 2015.

[8] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[9] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17):1–5, 2017.

[10] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

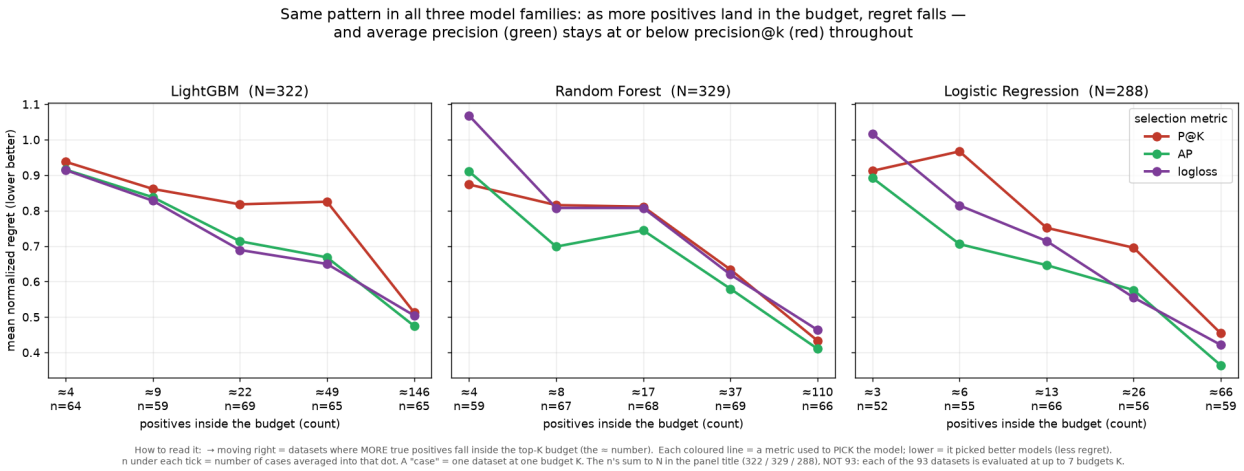

Figure 8: Regret per selection metric versus the number of true positives inside the budget, one panel per model family. In all three, regret falls as the count grows and average precision (green) stays at or below precision@k (red).

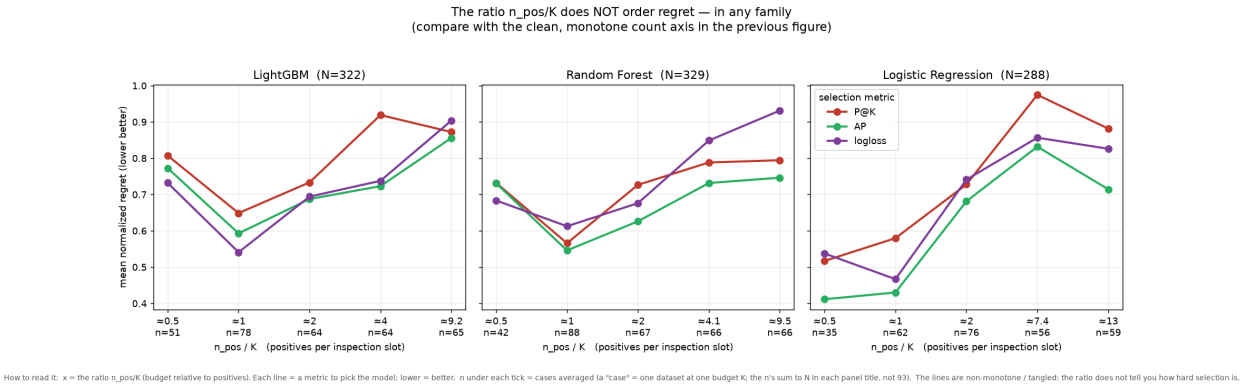

Figure 9: Regret versus the ratio n_{pos}/K, one panel per family. Unlike the count axis, the ratio does not order regret in any family — confirming that a scale-free ratio is the wrong lens.

[11] Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. OpenML: networked science in machine learning. *ACM SIGKDD Explorations Newsletter*, 15(2):49–60, 2014.

[12] Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7(91), 2006.

Figure 10: Regret versus the a-priori proxy $\min(K, n_{\text{pos}})$, one panel per family. Knowable before training, it orders regret monotonically in all three, with average precision (green) at or below precision@k (red) throughout.